VALQUO_LIVE_AUDIT.md — cold audit #2: the live product and the trust infrastructure

Auditor: cold (no history with this codebase). **Date:** 2026-08-10. **Tree audited:** `origin/main` @ `7694a32`, in a fresh worktree. **Method:** read-only. Every finding below is either quoted code, a measured run, or a live HTTP read of the production API. Nothing was changed, fixed or committed except these three files.

> ***Read this first.*** *The primary checkout at* `C:\Users\donni\Downloads\valuation-tool` *is* ***265 commits behind*** `origin/main`*.* `withhold.py`*,* `track_meter.py`*,* `research_log.py`*,* `build_ledger.py`*,* `VALQUO_LEDGER.md` *and* `PAPER_TRACK_CONTRACT.md` *— six of the files this commission names —* ***do not exist in it****. Anyone reading that checkout is reading the project as it was before the whole trust layer landed.* `git pull` *before anything else.*

---

1. Honest summary

The three worst things

1. The number one name on the public hot list carries a +203.6% fair value that the product's own valuation engine refuses to publish — right now, today. `KSPI` is rank 1 of 594 on `/api/hotstocks` with `fair_value: 274.13` against a `$90.30` price and `fair_value_withheld: false`. I ran the engine's own decision against the same name during this audit: it returns `publish: False`, `ratio: 5.64`, *"Cannot value this name: the model's $530.14 is 5.6x the $93.97 price."* KSPI is not an incidental example — it is **the** name the entire withholding subsystem was built for and is quoted by name in `withhold.py`'s own docstring. The refusal machinery is correct; it simply did not record a refusal for this row, and the peer estimator then filled the empty cell. Nothing in the product can currently see that this happened. (LA1)

2. The one record the project says cannot be re-derived is not in its backup. The `track-backup` workflow is titled *"the one thing that can't be re-derived"*. The committed artifact it produces, `data_export/paper_track_history.json`, records `"ingested_index_days": 0` and `ingested_index_track: null` — **the contract-bound Valquo Index series is absent from it entirely**. What *is* backed up is the Tradier sandbox book, which `PAPER_TRACK_CONTRACT.md` §5b registers as a separate experiment that may never be quoted as the Index. The workflow's anti-regression guard (*"Refuse to overwrite a real backup with an empty one"*) counts rows in the sandbox CSV only, so it cannot see the omission. And the README the exporter writes still labels the sandbox file *"daily Valquo Index vs SPY"* — the exact conflation that put a false performance claim into Discord on 2026-08-05, corrected in the module docstring and not in the file the module emits. The bound series' only copy is two rows in a gitignored CSV on one laptop. (LA2)

3. The live-track "Alpha / yr" and "Sharpe" are annualised on the number of rows recorded, not on time elapsed — and the recorder is missing 71% of its rows. `index_track.summarize` uses `days = len(series)` for both the `MIN_LIVE_DAYS` gate and the `252/days` exponent. `track_meter.gap_report` on the identical series says 7 expected trading days, 2 present, 28.6% coverage. Measured on one synthetic year with identical underlying returns: complete series → `ann_alpha 11.08%`, `sharpe 0.54`; the same year recorded at 33% → `ann_alpha 39.21%`, `sharpe 1.03`. Both the module docstring ("*No annualising a stub*") and the UI copy ("*Annualised figures are withheld until 60* ***trading days***") assert the guard is in trading days. The suite's fixture for that guard feeds a gapless series, so it has never been shown the input it exists to refuse. (LA3)

Overall state

This is, by a wide margin, the most carefully self-audited codebase I have read. The publication decision is genuinely consolidated to one function with one band and a test that fails if you restate it. `test_guards.py` is a real M3 census with an honest XFAIL mechanism that does not go green on a guard that cannot fire. The owner/demo/public split is a pure function pinned by a route-map walk. The whole suite is green: **26 suites, ~1,200 tests, 0 failures, 1 declared XFAIL** (an options-lane OI guard, correctly routed).

The failures are not in the reasoning. They are all in one place: **the seam between a correct in-process computation and the thing that actually ships it.** Four of my top five findings are the same shape —

- `run_scan` computes a `health` block precisely to surface dead themes and a broken refusal screen; `ci_scan.py` posts a `params` dict that omits it, so it is `null` on the live API (LA5);
- `screen._enrich_with_dcf` records refusals correctly; its `except: continue` has no counter, and `health.refusal_screen` excludes the top-N rows by construction, so the one place a refusal failure matters most is the one place nothing counts it (LA1);
- `track_export.payload` captures both forward records correctly; the artifact it produced contains one of them, and the guard checks the wrong one (LA2);
- `track_meter` models gaps rigorously; `index_track`, reading the same file, does not know gaps exist (LA3).

The project's own recurring-class list already contains this shape ("fixed field lists dropping computed values", "guards that cannot see"). What it has not yet named is the *generalisation*: **every one of these guards is correct in-process and blind at its own output boundary.** The verification effort has been aimed at the computation. Nothing systematically tests that what the computation decided is what leaves the machine.

Two structural notes. First, several findings below are already tracked (`V2F`, `PT-WRITER`, `PT-SPLIT`, and HANDOFF_STATUS's "BUGS FOUND" 2 and 5); I have re-verified them rather than re-discovered them and say so in each. Second, the project's habit of writing the correction into a docstring while leaving the shipped string alone has now happened three times in files I read (LA2, LA11, LA13) — a corrected comment beside an uncorrected artifact is a *worse* state than an uncorrected comment, because the next reader stops looking.

What I could NOT check, and why

- **The production environment.** I can read `config.py`'s defaults and the workflow files; I cannot see Render's or GitHub Actions' actual secret and env values. LA9 in particular (`TRADIER_TOKEN` absent from the hot-scan job) is marked HYPOTHESIS for exactly this reason.
- **Owner-only surfaces as served.** `/api/index-track`, `/api/track`, `/api/valquo-index` are owner-gated, so I audited the code path and the local data rather than the live response. LA3's numbers are measured against the real bound track files and a synthetic year, not against production.
- **Whether the Cowork writer `valquo-daily-track-write` exists.** It is on another machine. The ledger's `PT-WRITER` row records the same limit. What I can say is that the copy reachable from here still holds two rows, dated 2026-07-31 and 2026-08-06.
- **Anything needing licensed data.** `data/backtest` (Sharadar) and the ThetaData cache were not read; their absence from git is correct and is not a finding.
- **The options tree** (`edge/options_*`, ~11k lines) beyond its guard census. Out of the commission's scope and covered by its own lane.
- **Re-adjudicating any research verdict.** Explicitly out of scope; I audited only whether code enforces them.

Claim verification — 34 load-bearing claims sampled, 20 current (59%), 14 wrong or stale

Sampling was deliberately **biased toward load-bearing and mechanically checkable claims**, so this rate is not comparable to a random sample and is lower than the project's earlier exercise (43/62 = 69%) partly for that reason. Of the 14 misses, **9 are documentation or file:line rot and 5 are behavioural** — the behavioural ones are LA2, LA3, LA4 and LA13 below.

#	Claim	Source	Verdict
1	live `sector_neutral` defaults **true**	CLAUDE.md B7/G	**WRONG** — `config.py:165` is `false`; fixed since
2	live `residual_momentum` defaults **true**	CLAUDE.md B7/G	**WRONG** — `config.py:166` is `false`

#	Claim	Source	Verdict
3	`build_frame(metrics)` at `screen.py:256`	CLAUDE.md B7/G	**WRONG** — line 239
4	`rule_fired` at `fundamental_panel.py:3048`	CLAUDE.md B8	**WRONG** — line moved
5	tests are "16/16"	CLAUDE.md	**WRONG** — `test_edge.py` alone is 262/262; ~1,200 total
6	`low_risk` weight is 0	CLAUDE.md / settings	current
7	`insider` stays 0.125	CLAUDE.md	current
8	`sentiment` weight 0	settings.py	current
9	`sector_neutral` stays OFF	CLAUDE.md	current
10	one band, every surface imports it	publication.py	current — verified by identity across 4 modules
11	`withhold._band()` == `pipeline.FV_BAND_HIGH`	withhold.py:147	current
12	equity trial count `N` stays 131	HANDOFF_STATUS	current — `research_log.detail()` → 131
13	42.9% of deployed weight inert live	HANDOFF_STATUS / V2F	current — confirmed structurally
14	served payload's `health` is null	HANDOFF "BUGS FOUND" (2)	current — confirmed live
15	`test_saas.py` writes 2099 into the real DB	HANDOFF "BUGS FOUND" (5)	current
16	roth `measured.net_alpha` = 0.1163	settings.py	current
17	`DEFAULT_BOOK_CONFIG` = roth	settings.py	current
18	meter inception 2026-08-10 (vintage 2)	track_meter	current
19	meter verdict date 2031-08-10	track_meter	current
20	meter sigma 3.985 pp/month	track_meter	current — 3.9847
21	sandbox's 10% weights "violate the 8% cap"	track_meter docstring:75	**WRONG** — retracted by HANDOFF session 16
22	"no annualising a stub … `MIN_LIVE_DAYS` **trading days**"	index_track docstring	**WRONG** — counts recorded rows (LA3)
23	"withheld until 60 **trading days**"	static/app.js:1968	**WRONG** — same (LA3)
24	"the backup cron is a no-op if the primary landed"	auto-scan.yml:33	**WRONG** — 2026-08-07 *and* -08-08 both exist (LA4)
25	"BOTH forward records are captured"	track_export docstring:41	**WRONG in the artifact** — `ingested_index_days: 0` (LA2)
26	`paper_track_index.csv` = "Valquo Index vs SPY"	data_export/README.md	**WRONG** — it is the sandbox book (LA2)
27	a new `/api` route in neither list fails the suite	surfaces.py docstring:63	**WRONG** — two routes are exempted in the test (LA13)
28	holidays computed, "does not expire in a year"	market_session.py:88	current
29	freshness age is in trading days	freshness.py:10	current
30	not modelling holidays avoids "crying wolf"	freshness.py:28	**WRONG** — it makes the badge fire *sooner* (LA7)
31	the bound series has no automated writer	ledger `PT-WRITER`	current — still 2 rows
32	a demo session is read-only under every flag	surfaces.py	current — verified both directions
33	`MIN_LIVE_DAYS = 60`	index_track.py:44	current
34	ledger = 134 audit + 28 out-of-band rows	test_build_ledger	current — 162 rows

Guard coverage — which guards have a known-bad fixture, and which do not

`tests/test_guards.py` is a real census: 36 tests across three tiers, each feeding a guard the bug it exists to catch *and* a clean input it must not trip. It reaches the publication guard, the public-row NaN guard, `withhold_derived_figures`, the

screener lens (CHTR + negative yields), `record_refusal`, `results_file`'s block and schema checks, `signal_coverage`, the options coverage/sanity blocks, the vendor pricer check and the whole OI/remine/alias family. Its `UNTESTABLE` list is specific rather than a shrug, and its own xfail mechanism is tested.

Guards in the audited trees with NO known-bad fixture — protected by prose only:

Guard	Where	What is untested
`index_track` annualisation floor	index_track.py:491	never fed a **gapped** series; `test_screener.py:575` builds consecutive dates only. This is LA3
`index_track.MAX_PLAUSIBLE_SHARPE`	index_track.py:57	no fixture drives a degenerate low-variance series into it
`freshness.status`	freshness.py:49	no fixture where `as_of` is a **weekend or holiday**; both are live states (LA4, LA7)
`_enrich_with_dcf` fail-open branch	screen.py:395	no fixture asserts that a raising `value_ticker` is *counted*; there is no counter to assert on (LA1)
`health.refusal_screen`	screen.py:404	no fixture asserts it covers the top-`run_dcf_top` rows; by construction it does not
track-backup empty-backup guard	.github/workflows/track-backup.yml:78	shell in a workflow, no test at all; and it checks the wrong series (LA2)
`ci_scan` ingest payload	scripts/ci_scan.py:124	nothing asserts the posted `params` carries what `run_scan` computed (LA5)
`broker_fundamentals` EV==0 bank sentinel	broker_fundamentals.py:239	documented with measured evidence, no fixture feeding `enterprise_value: 0`
`_theme_contribution` dead-theme detector	screen.py:90	`test_theme_health` covers the meter, not this; and its output is discarded by LA5

2. Findings

Severity: **BLOCKING** = a wrong number is reaching a reader now · **HIGH** = a record corrupts or a published claim is materially wrong · **MEDIUM** · **LOW**.

LA1 — BLOCKING — The live hot list's #1 name publishes a +204% fair value the engine refuses

Files: `valuation/screener/screen.py:343-431` · `valuation/screener/fairvalue.py:246-299` · `scripts/ci_scan.py:90-135` **Class:** guards that cannot see / fail-open with no counter

What is wrong. `_enrich_with_dcf` correctly asks `publication.decide` and calls `record_refusal` when the model refuses. Its exception handler is `except: continue` (screen.py:395) — deliberate, documented fail-open. But **nothing counts a fail-open**. `_screen_refusals` reports `{"screened": n, "refused": after - before}` and is invoked on `rows[run_dcf_top:refusal_screen]` (screen.py:274) — i.e. **the top** `run_dcf_top` **rows are excluded from the counter by construction**, and production runs `SCAN_DCF_TOP=12`. So a fetch failure on rank 1 leaves no trace anywhere. `estimate_fair_values` then reads the untouched `fair_value: None` as "no DCF yet" and substitutes a peer estimate — the exact substitution `record_refusal` exists to prevent.

Evidence (live, 2026-08-10).

`GET /api/hotstocks?top=3` → `scan_date 2026-08-08`, `scored 594`, rank 1:

```
        ticker KSPI · price 90.30 · fair_value 274.13 · hot_score 100.0
```

`GET /api/whatdo?ticker=KSPI` →

```
        rank 1 · price 90.3 · fair_value 274.1343244549422 · upside 2.035817546566359
        fair_value_method "blended" · fair_value_withheld false · fair_value_withheld_reason null
```

`fair_value_method: "blended"` is proof the DCF pass wrote nothing: a published DCF is tagged `"dcf"` and a recorded refusal is tagged `"withheld"` (fairvalue.py:264, 271).

The engine's own verdict for the same name, run during this audit (`value_ticker("KSPI", CONFIG)` → `publication.decide`):

```
        price            : 93.975
        currency/fin ccy : USD / KZT | fx_rate: 0.002143 | fx_unresolved: False
        blend.value      : None
        withheld_value   : 530.1409151330242
        base_fair_value  : None
        verdict.publish  : False | ratio: 5.641297314530719
        verdict.reason   : Cannot value this name: the model's $530.14 is 5.6x the $93.97 price.
                           That gap is a data problem (currency or share count), not an opportunity,
                           so no fair value is published.
```

So the refusal is clean, deterministic and reproducible — it is not a borderline case. The 5x band on the *served* value cannot catch this: 274.13/90.30 = 3.04x, comfortably inside it. That is precisely the hole `_screen_refusals`' own docstring predicts ("*a refused 11x model is replaced by a 3.2x peer estimate that sits comfortably under it*") — and the screen that was built to close it does not cover the rows most likely to be read.

Blast radius. The most prominent row on the product's most public surface, on two endpoints, plus anything downstream of the snapshot (the Index book construction reads the same rows). A reader sees `+204% upside` on a name whose own valuation page says "Cannot value this name". This is the failure the entire `withhold.py` / `publication.py` / `record_refusal` programme exists to prevent, occurring on its own flagship example.

Cheapest verification / refutation. Fix LA5 first (one dict key), then read `health.refusal_screen` on the live payload: `screened: 0` or an unreported error count on a scan serving 594 names is the tell. Independently: run `python -c "from valuation.screener.screen import _enrich_with_dcf"` against a row for KSPI and confirm `record_refusal` fires; it does. The open question is only *why* it did not fire in CI, and the answer is unobtainable today because nothing logs it.

Note on scope. I am reporting the observable defect, not prescribing the fix. But the minimal change that makes this class visible is a third counter — `errors` — beside `screened` and `refused`, and extending the screen to `rows[:refusal_screen]` rather than `rows[run_dcf_top:refusal_screen]`.

LA2 — HIGH — The contract-bound forward track is absent from its own backup, and the guard cannot see it

Files: `.github/workflows/track-backup.yml:70-88` · `valuation/edge/track_export.py:126-172, 275-303` · `data_export/paper_track_history.json` · `data_export/README.md:16` **Class:** two recorders / two sources of truth + guard that checks the wrong object

What is wrong. Three compounding problems in the one mechanism the project calls irreplaceable.

(a) The bound series is not in the artifact. `payload()` *does* read the ingested Cowork tracker (`track_export.py:155-159`), so the code is right. The committed artifact is not:

```
        "counts": { "index_days": 4, "index_holdings": 10, "ingested_index_days": 0,
                    "option_alerts": 3, "paper_orders": 3, ... }
        "ingested_index_track": null
```

The four `index_days` that *are* backed up are the Tradier sandbox book — `data_export/paper_track_index.csv` rows dated 2026-08-03 … 08-07, `n_positions: 10` — which `PAPER_TRACK_CONTRACT.md` §5b registers as a separate experiment whose return series "is evidence about nothing the contract binds".

(b) The anti-regression guard checks the wrong file. The workflow step *"Refuse to overwrite a real backup with an empty one"* compares row counts of `data_export/paper_track_index.csv` only:

```
NEW=$(python -c "...csv.DictReader(open('data_export/paper_track_index.csv'))...")
OLD=$(git show HEAD:data_export/paper_track_index.csv | tail -n +2 | grep -c .)
if [ "$NEW" -lt "$OLD" ]; then ... exit 1; fi
```

`ingested_index_days` is never compared. The failure the guard was written for — *"the service comes up against a FRESH disk … and a well-behaved backup faithfully commits nothing over months of record"* — is therefore undetectable for the series that matters most, and has in fact already occurred silently (it is at 0 today).

(c) The emitted README mislabels it. `track_export.py:26-33` records the correction in prose: "`paper_track_index.csv` **was described here** as 'the daily Valquo-Index-vs-SPY series'. It is not." But `_README` — the string the code *writes* into `data_export/README.md` on every run — was not corrected, and the committed file reads:

```
| `paper_track_index.csv` | daily Valquo Index vs SPY, cumulative since inception |
```

This is the identical conflation that produced the false "+0.18 pp, Index beating SPY" Discord post on 2026-08-05, now preserved in the restore instructions.

Blast radius. The bound Valquo Index series exists in exactly one place reachable from here: `data/valquo_track_history.csv`, two rows, gitignored, on a single machine whose backup drive is recorded as hardware-dead. If that laptop dies, the record the signed five-year contract is measured on is gone, and the restore path a future reader follows points at the wrong book with a label that says it is the right one.

Cheapest verification. `python -c "import json;print(json.load(open('data_export/paper_track_history.json'))['counts'])"` — already done above. Then `git log --oneline -- data_export/` to confirm no commit ever carried a non-zero `ingested_index_days`.

LA3 — HIGH — The live track annualises on rows recorded, not on time elapsed

Files: `valuation/screener/index_track.py:440-502` · `valuation/web/static/app.js:1957-1969` · `valuation/web/hero.py:72-85` · `tests/test_screener.py:575-611` **Class:** two sources of truth (one module models gaps, its sibling does not) + a fixture that only exercises the clean case

What is wrong. `summarize()` sets `days = len(series)` (index_track.py:461) and uses it for both gates and for the exponent:

```
if days >= MIN_LIVE_DAYS and cum_v is not None and cum_s is not None:
    gv = (1.0 + cum_v / 100.0) ** (TRADING_DAYS / days) - 1.0
    gs = (1.0 + cum_s / 100.0) ** (TRADING_DAYS / days) - 1.0
    live["ann_alpha"] = gv - gs
```

`len(series)` is the number of rows the recorder wrote, not the number of trading days elapsed. `_daily_returns` chains cumulative levels, so a missing day silently produces a multi-day "daily" return — inflating the Sharpe by ~√k and over-annualising the alpha by the ratio of elapsed days to recorded rows.

Evidence — measured, not argued.

The real bound track, through both modules:

```
index_track.summarize(...)        -> live.days = 2   ("Live track is 2 trading days old")
track_meter.gap_report(...)       -> expected 7, present 2, missing 5, coverage 0.286
missing: 2026-08-03, -08-04, -08-05, -08-07, -08-10
```

One synthetic year, identical underlying daily returns, identical final cumulative levels, thinned three ways:

series	reported `days`	`ann_alpha`	`sharpe`
complete, 252 rows	252	**11.08%**	**0.54**
every 2nd day (50%)	126	24.05%	0.83
every 3rd day (33%)	84	**39.21%**	**1.03**

At the observed 28.6% recording rate the published annualised alpha would be inflated roughly **3.5×** and the Sharpe roughly **1.9×**, on the same year. The direction is *magnifying*, not merely *flattering* — a bad year would be published as far worse.

The stated protection is the thing that fails. The module docstring's rule 2 is "*No annualising a stub … Annualised alpha and Sharpe are only computed once there is enough history*". The UI prints, verbatim: *"Annualised figures are withheld until 60 trading days — compounding 2 days to a yearly rate would invent a number."* Both sentences describe a guard measured in trading days. The guard is measured in rows. `MIN_LIVE_DAYS` on rows is *conservative* in wall-clock (a gappy track takes longer to reach 60), which is exactly why the defect survives: the gate looks safe while the arithmetic behind it is wrong.

`hero.py:75` forwards the same `days` to the landing band, and the `note` string calls it "trading days".

Blast radius. `/api/index-track` (owner + demo/recruiter sessions, and everyone if `OWNER_SPLIT=false`) and the landing hero. Not currently drawn — `days = 2` is far below the floor, so both figures render "—" today. It becomes live the moment the track has 60 recorded rows, which at the current rate is roughly one calendar year away, and by then the sentence "withheld until 60 trading days" will have been true for a year and will read as evidence the guard works.

Cheapest verification. `track_meter.gap_report` and `index_track.summarize` on the same series disagree about how many days the track has (7 vs 2). Any fix must make `days` a function of `gap_report`'s elapsed count, and the fixture at `test_screener.py:580` must be given a gapped series.

LA4 — HIGH — The daily snapshot is stamped after the scan, so the backup cron dates it the next day — and Saturday reads as "last close"

Files: `valuation/screener/screen.py:328,434-435` · `.github/workflows/auto-scan.yml:31-34` · `valuation/saas/app_saas.py:493-508` · `valuation/screener/freshness.py:49-74` **Class:** clock at the wrong end of a long operation + a guard that never validates its own input

What is wrong. `run_scan` computes `scan_date = _today()` at **line 328 — after** the entire scan (universe fetch, ~800 metric fetches, DCF on the top 12, a 500-name refusal screen; the workflow allows it 60 minutes). The backup cron fires at **23:41 UTC**, nineteen minutes before midnight. Any backup run longer than nineteen minutes is stamped with the **next calendar day**.

Evidence (live). `GET /api/hotstocks` → `history`:

```
["2026-08-08","2026-08-07","2026-08-06","2026-08-05","2026-08-04",
 "2026-07-29","2026-07-28","2026-07-27","2026-07-26"]
```

`2026-08-08` is a **Saturday**. `2026-07-26` is a **Sunday**. And **both** `2026-08-07` **and** `2026-08-08` **exist for the single Friday close** — the primary cron landed as Friday, the backup crossed midnight and landed as Saturday.

Three consequences:

1. **The backup cron is not a no-op.** `auto-scan.yml:33` states: *"The ingest endpoint is

idempotent per day, so the backup is a no-op if the primary already landed."* The idempotency key is `f"hot_processed_{scan_date}"` (app_saas.py:494). Two different `scan_date`s → two different keys → the `if not already` block fires twice: `tracker.log_hot` writes a **second forward-track pick row for the same close**, and `notify.post_hot_digest` posts the Discord hot digest **twice**. The forward hot10 track therefore double-counts Friday 2026-08-07.

2. **The ranking is misdated.** The served snapshot is Friday's close labelled Saturday.

3. **The staleness guard endorses it.** `freshness.status` never checks that `as_of` is a trading

day. Measured: `status("2026-08-08", today=date(2026,8,10))` returns `level: "fresh"`, `message: "As of 2026-08-08 (last close)."` There was no close on 2026-08-08.

Blast radius. Every scan-derived surface carries the wrong date and a "fresh" badge that endorses a non-existent close; the forward hot10 record — a track whose entire value is being untainted — gains duplicate rows; Discord readers get the digest twice on backup-cron days.

Cheapest verification. Already done: two adjacent dates for one Friday close in the live `history`, plus a Sunday two weeks earlier. Locally: `python -c "from valuation.screener.freshness import status; import datetime as d; print(status('2026-08-08', today=d.date(2026,8,10)))"`.

LA5 — HIGH — `ci_scan` drops the `health` and `filtered` blocks, so every data-health signal the scan computes reaches nobody

Files: `scripts/ci_scan.py:124-126` · `valuation/screener/screen.py:280-331` · `valuation/web/app.py:516-517`
Class: fixed field list dropping computed values **Status:** already recorded as HANDOFF_STATUS "BUGS FOUND (2)"; re-verified and traced to the line.

What is wrong. `run_scan` builds a `health` dict containing `theme_coverage`, `theme_contributing` (the post-standardisation dead-theme detector), `refusal_screen`, `display_coverage`, `fundamentals` and `api_budget`, and a `filtered` audit of every rejected name with reasons and examples. `screen.py:288-297` states plainly why: *"a silent zero here is exactly how the gap survived unnoticed."*

`ci_scan.py` then posts:

```
        _post("/admin/ingest-snapshot", {
            "scan_date": res["scan_date"], "provider": res.get("provider", "ci"),
            "rows": rows, "params": {"scope": scope, "universe_size": res.get("universe_size")}})
```

`health` and `filtered` are not in the dict. `/api/hotstocks` serves `"health": params.get("health")` and `"filtered": params.get("filtered")` — both `null`.

Evidence (live). `GET /api/hotstocks` → `health` is `null`, `filtered` is `null`, on a payload reporting `universe_size: 800, scored: 594`. The `run_scan` return value *does* contain both; `ci_scan` prints several of them to the Actions log (lines 107-121) and then discards them at the one boundary where they would persist.

Blast radius. This is the mechanism by which LA1 and LA6 are invisible. `refusal_screen` exists specifically so that *"a silent zero here is the tell that Bug B is back"*; nobody can read it. `theme_contributing` exists specifically to distinguish "the column is full" from "the theme moves the score" — the distinction on which the 42.9%-inert finding rests; nobody can read it. The scan's own instrumentation is complete and its output is thrown away in transit.

Cheapest verification. Add `"health"` and `"filtered"` to the posted `params` dict and re-read `/api/hotstocks`. One line.

LA6 — MEDIUM — 42.9% of the deployed composite weight reaches no live score, and only the discarded block can tell

Files: `valuation/screener/factors.py:254,267,270-273` · `valuation/screener/providers.py:160-163` · `valuation/screener/settings.py:75-80` · `scripts/ci_scan.py` (no insider enrichment) **Class:** silently-empty inputs **Status:** tracked as ledger `V2F`; re-verified independently, with one addition.

Verified structurally, not just re-quoted:

- `capital_discipline` = `mean(z_neg_issuance)`; `neg_issuance` = `-share_issuance`; `providers.company_to_metrics:162` hard-codes `"share_issuance": None` and `_fmp_to_metrics` never sets it. **No live source exists.** Weight 0.125.

- `institutional` = `mean(z_inst_accum, z_sm_breadth)`; neither key is emitted by `company_to_metrics`, `_fmp_to_metrics` or `broker_fundamentals.to_metrics`. **No live source.** Weight 0.125.
- `insider` = `0.0` constant when `insider_score` is absent (factors.py:273). `screener/insider.py::enrich_insider` is invoked **only** by the `--insider` CLI flag (`scan.py:45`); `scripts/ci_scan.py::run_hot` never calls it. Weight 0.125.

3 × 0.125 = 0.375 of a 0.875 total = **42.9%**, matching the greeks lane's measurement on 500 served rows.

My addition, which is the part that matters for tooling. The three fail *differently* and only one instrument distinguishes them. `capital_discipline` and `institutional` arrive **null** — `theme_coverage` catches those. `insider` arrives **100% non-null with one distinct value** — `theme_coverage` reports it as fully covered, and only `theme_contributing`, which z-scores first, reports 0. That is the block LA5 discards. So the single most deceptive of the three dead themes is detectable by exactly one number, and that number never leaves the runner.

Blast radius. The public hot list is a four-theme book (value, quality, size, momentum) wearing the weights of a nine-theme one. No claim is made here about the return cost — that is a backtest question and out of this audit's scope.

LA7 — MEDIUM — `freshness` neither validates its input date nor models holidays, and its docstring has the direction backwards

Files: `valuation/screener/freshness.py:10-37,49-74` **Class:** guard that cannot see

Two defects and one inverted justification:

1. **No trading-day validation on `as_of`.** A Saturday-dated snapshot returns

`level: "fresh"`, `message: "As of 2026-08-08 (last close)."` (measured — see LA4). The module is the product's single defence against "stale data presented as fresh", and it will endorse a date on which no close existed.

2. **Holidays unmodelled.** `status("2026-12-24", today=date(2026,12,28))` returns

`age_trading_days: 2`, counting Christmas Day as a trading day.

3. **The stated justification is backwards.** freshness.py:28 reads: *"Holidays are not modelled —

the cost of being one day generous around Thanksgiving is far lower than the cost of crying wolf." *Not modelling holidays makes the computed age **larger**, so the badge fires **earlier** — that* is* crying wolf. A correct holiday model would make it more generous, not less.

`market_session.market_holidays` already computes the NYSE calendar and is imported by `track_meter`. `freshness` does not use it — a second, weaker calendar in the same repository.

Blast radius. Low individually (a badge), but it is the guard that turns LA4's misdated snapshot green, so it is load-bearing for a finding above it.

LA8 — MEDIUM — "Days" on the forward-track cards is a row count presented as trading days

Files: `valuation/screener/index_track.py:474,504-506` · `valuation/web/hero.py:75` · `valuation/web/static/app.js:1960`

The UI renders `metric("Days", live.days)` beside "Alpha / yr" and "Sharpe", and the server's own `note` reads *"Live track is 2 trading days old — far too short to judge."* The track is **7 trading days old** and has 2 recorded rows. A reader is told the record is short when the real statement is that the recorder is missing 71% of its rows — a different problem with a different owner (ledger `PT-WRITER`, Cowork lane). The one number that would make the recording failure visible on the surface where someone would act on it is being spent to say something else.

Separately from LA3: even before annualisation is reachable, this sentence is false today.

LA9 — MEDIUM (HYPOTHESIS) — The scheduled hot scan may run with no broker token, and nothing observable would say so

Files: `.github/workflows/auto-scan.yml:96-118` · `valuation/screener/providers.py:208-226,266-280`

The `hot` job's `env:` block passes `BASE_URL`, `ADMIN_TOKEN`, `ANTHROPIC_API_KEY`, `DISCORD_WEBHOOK_URL`, `FMP_API_KEY` and `SEC_USER_AGENT`. It does **not** pass `TRADIER_TOKEN` (the `intraday` job does). `CONFIG.tradier_token` reads only that env var, and both `broker_universe.available()` and `broker_fundamentals.available()` are `bool(cfg.tradier_token)`.

If the secret is genuinely absent from that job, then on every scheduled hot scan: `prefetch` returns `{}` with note *"no TRADIER_TOKEN — broker fundamentals unavailable"*, and `_broker_universe` returns `[]` with note *"no TRADIER_TOKEN — cannot source the broker universe; falling back"* — i.e. the universe comes from the SEC EDGAR filer list (no price, no market cap, no size ordering) truncated to the first 800, and every name runs on the rate-limited per-name free stack alone. That is not the path the workflow comment describes (*"whole_market now resolves to the broker's liquidity-ranked universe (~7,100 listed names, kept to the most liquid SCAN_LIMIT)"*).

Marked HYPOTHESIS because I cannot read Actions' secret scope, and because a job-level `env:` is the only place I can look. The observable that would settle it in one read — `health.universe_note` and `health.fundamentals.broker.note` — is precisely what LA5 discards. The two findings interlock: this one is *unfalsifiable from outside* until LA5 is fixed.

Cheapest verification. Open the most recent `hot` run's log in Actions and look for the `universe:` line `ci_scan.py:109` prints, or add `TRADIER_TOKEN: ${{ secrets.TRADIER_TOKEN }}` to the hot job and compare `scored` before/after.

LA10 — LOW — A withheld row keeps its method and confidence labels

Files: `valuation/web/withhold.py:168-206` · `valuation/screener/fairvalue.py:293-297`

In `/api/hotstocks` the order is `estimate_fair_values(rows, …)` then `withhold_implausible_fair_values(rows)`. The first sets `fair_value_method` (e.g. `"blended"`) and `fair_value_confidence` (e.g. `"medium"`); the second blanks `fair_value` and `upside` and sets `fair_value_withheld`, but leaves the two labels in place. A withheld row therefore ships as `{fair_value: null, fair_value_method: "blended", fair_value_confidence: "medium", fair_value_withheld: true}` — describing the confidence of a number that is not there. Cosmetic today (no surface renders it), but it is the same "a label survives the value it described" shape the module exists to eliminate, and a future renderer would pick it up.

LA11 — LOW — `track_meter`'s docstring still carries a diagnosis the project retracted

Files: `valuation/edge/track_meter.py:73-76`

> *"It does NOT bind the Tradier sandbox engine in `paper_track.py`: that engine records a DIFFERENT book (10 names, equal-weighted at 10%, which the contract's own 8% cap forbids)"*

HANDOFF_STATUS session 16 (2026-08-10) retracts this explicitly: *"That is not a cap violation. The code sets the cap to `max(8%, 1/n)` on purpose, because ten names at 8% sum to 80%. The weights were correct for the book."* The **conclusion** (the engine is not the Index) survives on the corrected ground (book *size*, 10 vs 86), but the reason given here is one the project has disproved. The same retracted sentence appears in `index_track.py:286` and `recap.py:19-21`.

Left uncorrected, a future reader who checks the cap will find it correct and may conclude the whole separation was mistaken.

LA12 — LOW — `sector_attractiveness.median_upside` mixes two populations in one row

Files: `valuation/screener/sectors.py:43` · `valuation/web/app.py:493-514`

`/api/hotstocks` computes `sector_attractiveness(all_rows)` on rows taken straight from the database — **before** `estimate_fair_values` has run on anything (it runs only on the served slice, `rows`). Only names that got a full DCF carry an `upside` in the stored snapshot, and production runs `SCAN_DCF_TOP=12`. So each sector's `median_upside` is a median over at most one or two names while `count` in the same object reports the full sector membership.

Not currently rendered (`app.js` reads `avg_composite` only), so this ships in the public payload and nowhere else. It becomes a wrong number the day someone draws it.

LA13 — LOW — `surfaces.py` claims a completeness property the suite does not enforce

Files: `valuation/saas/surfaces.py:59-65` · `tests/test_public.py:123-129,210`

The docstring's stated safety net:

> *"`PUBLIC_API` below records the other half explicitly, so the test can assert that every registered /api route is knowingly on one side or the other — a new route lands in neither list and fails the suite until someone decides."*

Walking the app's real URL map: `/api/option-alerts/open` and `/api/option-alerts/outcome` are in neither `PUBLIC_API` nor `OWNER_ONLY_PATHS`. `test_public.py:127` skips them (`if p.startswith("/api/option-alerts/"): continue # admin-token endpoints`).

The exemption is deliberate and correct — both handlers check `_admin_ok()` — but it lives in the test, not in the module that states the policy. The property as written in `surfaces.py` is not the property enforced, and the module has no record that a third category exists. A future admin-token route added under a different prefix gets neither the list nor the exemption.

LA14 — LOW — `market_holidays(y)` can return a date in year `y-1`

Files: `valuation/screener/market_session.py:85-107`

`_observed(date(y,1,1))` moves a Saturday New Year's Day to the **preceding Friday**, which is 31 December of `y-1`. Measured: `market_holidays(2028)` contains `2027-12-31`; `market_holidays(2033)` contains `2032-12-31`.

Inert for `is_trading_day`, which queries `market_holidays(d.year)` and therefore never sees it — and inert *correctly*, because the NYSE does not close on 31 December when 1 January falls on a Saturday. But it means the set is not what its name says, and any caller that iterates it (rather than testing membership) gets a date outside the year it asked for while the correct year's set silently lacks it. Nothing iterates it today.

LA15 — LOW — The test suite writes a year-2099 snapshot into the real screener database

Files: `tests/test_saas.py:200-205` **Status:** already recorded as HANDOFF_STATUS "BUGS FOUND (5)"; re-verified.

`test_saas.py` POSTs `/admin/ingest-snapshot` with `scan_date: "2099-01-01"` against a `Store()` resolving to the repository's real `data/screener.db`. `Store.latest_scan_date()` orders by `scan_date DESC`, so after any test run on a developer machine every scan-derived surface — `/api/hotstocks`, `/api/valquo-index`, `/api/whatdo`, the hero — reads the 2099 fixture as the latest scan. Freshness then reports it as a future date. Harmless in CI (fresh checkout), and it is why running the suite locally makes the app show test data.

---

3. Appendix — one page, clearly labelled untested speculation

Not a signal proposal (explicitly out of scope). One structural observation the findings converge on, offered as a hypothesis about *where to point the next verification effort*, not as a change:

Every finding above except LA6 sits at an output boundary, and none of the project's existing guard families watches one. `test_guards.py` verifies that guards fire on bad *inputs*. `signal_coverage` verifies inputs are present. `sanity_check` verifies inputs are sane. The publication consolidation verifies one *decision*. What no instrument covers is the assertion "*the thing that left this process is the thing this process computed*" — which is LA1 (a refusal computed, not recorded), LA2 (a record captured, not committed), LA3 (a series read, its gaps dropped), LA4 (a date computed at the wrong moment), LA5 (a health block computed, not posted).

A plausible fourth tier for the M3 census would be *transport* fixtures: for each boundary that serialises (`ci_scan` → `/admin/ingest-snapshot`, `payload()` → `data_export/`, `run_scan` → `save_snapshot`, `summarize` → `/api/index-track`), one test that computes an object with a known distinguishing field and asserts that field survives to the other side. Every finding above would have been caught by exactly one such test, and all five would be cheap — none needs a network, a vendor key or a panel.

Untested, unmeasured, and offered only because five independent findings landed on the same seam.

Read-only audit. No production code, ledger, handoff or register was modified. Deliverables: `VALQUO_LIVE_AUDIT.md`, `valquo_live_audit_items.json`, `VALQUO_LIVE_AUDIT.pdf`.